Full Stack Application Development Notes (Course Code: 23SDCS12A)

Introduction to Full Stack Development and Environment Setup

Introduction to Full Stack Development

What is Full Stack Development?

Full-stack development refers to the process of designing, building, and maintaining both the frontend (client-side) and backend (server-side) components of a web application. A full- stack developer possesses a broad skill set that spans multiple layers of the technology stack, including:

- **Frontend**: User interfaces, interactivity, and responsiveness.
- **Backend**: Server logic, APIs, and database management.
- **DevOps**: Deployment, scaling, and infrastructure management.
- **Detailed Explanation**:
  A full-stack developer acts as a bridge between the visual design that users interact with and the underlying systems that store and process data. This role requires proficiency in languages like HTML, CSS, JavaScript, and Java, as well as frameworks such as React and Spring Boot. Beyond coding, full-stack developers must understand system architecture, performance optimization, and user experience principles.
- **Importance in Industry**:
  The versatility of full-stack developers makes them invaluable in startups, small teams, and large enterprises alike. They can take a project from concept to deployment, reducing the need for multiple specialized roles. According to industry reports (e.g., Stack Overflow Developer Survey), full-stack developers consistently rank among the most sought-after professionals due to their ability to handle end-to-end development.
- **Career Prospects**:
  The demand for full-stack developers has surged with the rise of web-based applications. Job titles such as "Full Stack Engineer," "Web Developer," or "Software Engineer" often command salaries exceeding $100,000 annually in tech hubs like Silicon Valley. Freelance opportunities and remote work are also abundant, given the global reliance on digital solutions.

Evolution of Web Development

The web has undergone significant transformation since its inception:

- **Early Web (1990s)**:
  Static HTML pages dominated, offering limited interactivity. Websites were primarily informational, built with basic tags like `<h1>`, `<p>`, and `<img>`. Example:

```
<html>
<body>
  <h1>Welcome to My Page</h1>
  <p>This is a simple website.</p>
</body>
</html>
```

  - **Challenges**: No dynamic content, manual updates required.

- **Dynamic Web (Late 1990s - Early 2000s)**:
  Server-side scripting languages like PHP, ASP, and CGI introduced interactivity. Databases (e.g., MySQL) enabled data-driven sites. Example:

```
<?php
$name = "User";
echo "<h1>Hello, $name!</h1>";
?>
```

  - **Advancements**: Forms, user logins, and content management systems (e.g., WordPress).

- **Modern Web (2010s - Present)**:
  Single Page Applications (SPAs) powered by JavaScript frameworks (e.g., React, Angular) emerged. RESTful APIs and microservices enabled scalable, modular architectures. Cloud computing and DevOps practices further revolutionized deployment. Example SPA structure:

```
function App() {
  return <div><h1>Modern SPA</h1></div>;
}
```

- **Historical Milestones**:
  - 1990: Tim Berners-Lee invents the World Wide Web. 1995: JavaScript introduced by Netscape.
  - 2003: Rise of AJAX for asynchronous updates.
  - 2010: React launched by Facebook, transforming frontend development.

Why React and Spring Boot?

- **React**:
  - **Component-Based Architecture**: Encourages reusable, modular code. Example:

```
function Button({ label }) {
  return <button>{label}</button>;
}
```

  - **Virtual DOM**: Optimizes rendering by minimizing direct DOM manipulation, enhancing performance.
  - **Ecosystem**: Rich libraries (e.g., Redux, React Router) and community support.
- **Spring Boot**:
  - **Convention Over Configuration**: Reduces boilerplate code, speeding up development.
  - **Embedded Server**: Built-in Tomcat allows standalone applications. Example:

```
@SpringBootApplication
public class MyApp {
  public static void main(String[] args) {
    SpringApplication.run(MyApp.class, args);
  }
}
```

  - **Enterprise-Ready**: Supports security, scalability, and integration with databases.
- **Synergy**: React's frontend flexibility pairs seamlessly with Spring Boot's robust backend capabilities, making them ideal for modern full-stack projects.

Subsections

- **History of Frontend Technologies**:

  From static HTML to CSS frameworks (e.g., Bootstrap) and JavaScript libraries (e.g., jQuery) to modern frameworks like React, Vue, and Angular.
- **History airtight of Backend Technologies**:

  Evolution from CGI scripts to PHP, Ruby on Rails, Django, and now Spring Boot and Node.js.

- **Why This Course Uses React + Spring Boot**:
  React's popularity in UI development and Spring Boot's dominance in enterprise Java applications provide a balanced, industry-relevant skill set.

Setting Up the Development Environment

Installing NodeJS and NPM

NodeJS is the runtime for JavaScript outside the browser, and NPM is its package manager. Below are step-by-step instructions for installing them on different operating systems.

- **Ubuntu**:
  - **Step 1**: Open your terminal by pressing `Ctrl + Alt + T` or searching for "Terminal" in the application menu.
  - **Step 2**: Update the package list to ensure you're installing the latest versions:

```
sudo apt update
sudo apt upgrade -y
```

    - `sudo` runs the command as an administrator, `apt update` refreshes the package index, and `apt upgrade -y` updates installed packages (the `-y` flag auto- confirms).
  - **Step 3**: Install NodeJS and NPM:

```
sudo apt install nodejs npm -y
```

- This command fetches and installs both NodeJS and NPM from Ubuntu's repositories.

- **Step 4**: Verify the installation to confirm it worked:

```
node -v  # e.g., v18.17.0
npm -v   # e.g., 9.6.7
```

- These commands display the installed versions of NodeJS and NPM, respectively.

- **Troubleshooting**:
  - If the versions are outdated or you need a specific version, install Node Version Manager (NVM):

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.5/install.sh | bash
```

  - Close and reopen the terminal, then install and use a specific NodeJS version:

```
nvm install 18
nvm use 18
```

  - Verify again with `node -v` and `npm -v`.

- **Windows**:

- **Step 1**: Open your web browser (e.g., Chrome, Edge) and navigate to nodejs.org (https://nodejs.org).
- **Step 2**: Download the LTS (Long-Term Support) version by clicking the "LTS" button. This version is stable and recommended for most users.
- **Step 3**: Locate the downloaded `.msi` file (usually in your Downloads folder) and double-click it to run the installer.
- **Step 4**: Follow the installer prompts:
  - Accept the license agreement.
  - Choose the default installation path (e.g., `C:\Program Files\nodejs\`).
  - Ensure "Add to PATH" is checked so you can use NodeJS from the command line.
  - Click "Next" through remaining steps and then "Install."
- **Step 5**: Open Command Prompt (press `Win + R`, type `cmd`, and hit Enter) and verify:

```
node -v
npm -v
```

  - You should see version numbers (e.g., `v18.17.0` for NodeJS).

- **Troubleshooting**:
  - If the commands aren't recognized, ensure NodeJS is in your PATH:
    - Right-click "This PC" > Properties > Advanced system settings > Environment Variables.

- Under "System variables," find "Path," click "Edit," and add `C:\Program Files\nodejs\` if it's missing.
- Reopen Command Prompt and retry.

- **MacOS**:
  - **Step 1**: Open Terminal (found in Applications > Utilities or via Spotlight with `Cmd + Space`).
  - **Step 2**: Install Homebrew, a package manager, if not already installed:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

    - Follow the on-screen instructions to complete Homebrew installation.
  - **Step 3**: Install NodeJS and NPM using Homebrew:

```
brew install node
```

    - This downloads and installs the latest stable version.
  - **Step 4**: Verify the installation:

```
node -v
npm -v
```

    - Check the version numbers to ensure success.
  - **Troubleshooting**:
    - If Brew fails, install Xcode command-line tools first:

BASIC HTML:

--> HTML(Hypertext Markup Language): -

- It is used to define the structure of web document.
- It is developed in 1991 by Tim BernersLee at CERN.
- Hypertext is used to specify document as text and markup language have some set of pre-defined tags, to customise web page components.

```
-------------------------------------
| Basic structure of html: -        |
| --------------------------------- |
|       <html>                      |
|       <head>                      |
|       <title>                     |
|       <body>                      |
|       </html>                     |
 -------------------------------------
```

--> Title Tag: -

- It the file what you save as the HTML file, that will be rendered as URL in web Browser

--> Body Tag: -

- It is the actual content display area.
- Under body tag we can use para tag - (p).
- If you want to break the lines in the paragraph use (br) tag.
- If you want specify the heading in the para, we can use heading tags from <h1> to <h6>

-- > If errors occurs in future: -
Follow the commands: -
- open windows+R tab
- press cmd and click enter
- type code . and click enter

Types of Application:

1. Static Web Applications --> HTML, CSS, JS
2. Dynamic Web Application --> Front End + Database (Server)
   JAVA --> JSP, Servlet + SQL/MySQL/Postgres
   PYTHON --> Flask + SQL/MySQL/Postgres
   .Net --> Php/c# + SQL/MySQL/Postgres

3. Full Stack Application

   Frontend + Backend + Database
   Python --> Django/Flask + Python + SQL/MySQL/Postgres
   Java --> React + Spring Boot + MySQL and MangoDB

Full Stack Application:

1. It should follows MVC (Model View Controller) architecture.
2. It should be Rest API.

SPRING BOOT:

1. Rest API backend framework
2. Maven Framework to create Spring Boot Project
    Maven project structure
        src/main/java --> implement
        src/main/resources --> all configurations
        src/main/test
        pom.xml
            add all dependencies

Spring Boot sterio Type:

1. entity (com.klu.entity)
2. model (com.klu.model)
3. service (com.klu.service)
4. controller (com.klu.controller)

Visual Studio

How to create react class component?

class component - Upper case 1 st letter

```
import react , {component} from react

export default class name extends component{

 render(){


  return , <h1></h1>

        <div></div>

       <button></button>


 }
}
```

Arrow function component

- it is having only variable.

const home --> arrow function component

scope of variable - const

```
const home = () --> {


 return {


        <div></div>
    }
}

export default home;
```

routing - dom router-->routing techniques

materia ui -- web component templates

npm install dom router

npm install app_name

default -- npm install

AXIOS

it is the package commonly used to install thirdparty web api's in frontend as well as backend applications based on the project requirements we can define configuration of those a api's via defined config.jsx or by axios client.jsx (vite app)

Axios client application intracts with third party api server through http request and http response via get and post methods GET nad post method , now to configure the axios client we have to specify 2parameters namely base url and headers with content type

Note:- The base url might be mock api url or database connection url or backend application url (spring boot)

npm install axios is the command to install the npm associated packages in react

Axios feature include json (java script object notation) parsing, request or response handlers error handling and axios have good compatibility with all types of the browses

The listed four features are manual intervention needed in fetch api's

The below syntax to configure the axios client in Reactapp

```
const axiosClient = axios.create
({
  baseURL: "https://jsonplaceholder.typicode.com", // Mock API URL
  headers:
 {
    "Content-Type": "application/json",
  },
});
```

Spring Suit Tools

jpi interface

jpi repository

--> auto

--> the annotation is used to fall validating multiple enterprise java ebc of the same type(write field, constructor, method parameters)

--->atvalue --> this annotation is used under resources folder and it is configure in application.properties

eg: data source.username = root

data source.passwd = root

-->atResponseBody :

In spring mbc (model view controller) application when a controller is invoked is processes the request in the form of http response

which will return the view which is actually rendered. At response body annotation is one at controller layer of spring mbc application.

--> AtRestcontroller :

Atrest controller is the combination of atcontroller and atresponse body. Atrest controller handles the request effectively and

produce the response (http response) without need of view template.

-->AtService Annotation:

service layer in spring web application encapsulate the business logic of your application and all responsible for preforming

assigned tasks and coordinating the actions between the components.

find by id --->pre defined function

1st phase : url

PO-XAMBALE FILE --> configure un these file

--> Annotations used in the jpa data with rest:

Stateful resource transfer -rest web service where track the sessions of users

cookies - it is stacking the client state information -- history, screentime etc... --> session tracking

--> At Entity: -

It is used in spring JPI data project dependency of your application.
This annotation is just defined one line above your class.

At IDE -- AT Generated Value (strategies) = Generation type.auto
At IDE annotation is declared or defined immediately after class

Note: -

click on source in spring tool application and select the objects that you declare in the class file to generate the constructions via getters and setters.

--> Repositary: (create Repositary)

At repositary annotation:

It is used to create interface for the spring web application in the we. we gave to use the keyword calls extends, that will list out proposals that are
needed for web projects.

--> Cross Origin: -

It is an API (application program interface).It is used to check wheather it is connected to frontend and backend of the web development for getting http
responce and request, for these we are using the annotation
cross origin.

MY SQL

My Sql is the relational database schema that is used as database using some familiar api's to deploy the web applications.

the familiar api used is jduc(java database connectivity)

jduc establish database connection between java application and database.

structured query language execution is done by queries statements stored in database.

data retrieval also done by jdvc and make it available for java applications and jdvc also preforms data manipulation.

hence jdvc is a java api that allows programs to connect to interact with the database and also provide standard interface for the java applications,

to access and manipulate the data in the relational databases (my sql server)

jdvc api have four set of key classes and interfaces

connection connects databases

statements represents sql statements (parameterized constructor), triggers and cursors

prepared statements represents pre complied sql statements (to identify the particular field in the table by using the database)

result set represents result of a query (by using the database. fetching the result and also output)

driver manager manages the registrations and loading of jdvc drivers.

application software and utility software.

eg: usb

its acts as interface and the java application - jdvc

using the driver manager class Connection Con=Drivermanager.get Connection(url,user name,pass)

Role Based Annotation Spring Boot

Roles are generally classified into user role admin role and manager role to grant access to the user and also restricts privaleges to
the user fimilearly or mostly used annotation is @rolesallowed (read ,write, execute permissions to enable authorised user only read, write or execute)
The spring boot have specialised features to enforce security and authorisation @enable method security have sub-annotations @pre-authorize @post-authorize
@pre filter @post filter this methods authorise method invocations input parameters and return values

Note:

Spring boot starter application does not activate method level authorization by default.
Role admin follows the role highracy to authorize users.
-- > Sample annotation or Syntax:

```
@Bean
static RoleHierarchy roleHierarchy() {
   return RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > permission:read");
|
}
```

```
@Bean
static RoleHierarchy roleHierarchy() {
   return RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > permission:read"); -role hierarchy
}
```

First, to use Spring Method Security, we need to add the spring-security-config dependency:

```
<dependency>
   <groupId>org.springframework.security</groupId>
   <artifactId>spring-security-config</artifactId>
</dependency>
We can find its latest version on Maven Central
(https://mvnrepository.com/artifact/org.springframework.security/spring-security-config).
```

```
XML :   <bean id="roleHierarchy"
      class="org.springframework.security.access.hierarchicalroles.RoleHierarchyImpl" factory-
method="fromHierarchy">
   <constructor-arg value="ROLE_ADMIN > permission:read"/>
</bean>
```

Java WEB TOken Security:

Java web token ensures authorized users have accessed the application or not jwt offers flexible and state less way to verify user identity and secure api endpoints.
jwt security: POST] /auth/signup → Register a new user[POST] /auth/login → Authenticate a user
[GET] /users/me → Retrieve the current authenticated user[GET] /users → Retrieve the current authenticated user.
The routes “/auth/signup” and “/auth/login” can be accessed without authentication while “users/me” and “users” require to be authenticated.

DOM Manipulations

In JavaScript, the **DOM (Document Object Model)** is a programming interface that allows scripts to access and manipulate the structure, style, and content of HTML documents. It represents the page as a tree of objects, where each node corresponds to a part of the document (such as an element, attribute, or text content).

Types in Dom manipulations:

Element accessing

↓

→getElementById()

→getElementsByTagName()

→getElementsByClassName()

→querySelector()

→querySelectorAll()

Element Manipulations

↓

→appendChild()

→removeChild()

→replaceChild()

→insertBefore()

→CloneNode()

Attribute Manipulation

↓

→getAttribute()

→setAttribute()

→removeAttribute()

→hasAttribute()

Event Handling

↓

→addEventListener()

→removeEventListener()

→dispatchEvent()

CSS manipulation

↓

→classList.add()

→classList.remove()

→classList.containes()

→classList.toggle()

Element Creation

↓

→createElement

→createTextNode()

→createDocumentFragment()

Element accessing

1. getElementById(): getElementById() is a widely used method in JavaScript for DOM (Document Object Model) manipulations. It allows you to select and manipulate HTML elements by their id attribute. Here's how it works and how you can use it:

Syntax:

```
document.getElementById('id')
```

Ex:

```
<body>
     <h1 id="id1"></h1>

   <script>
       document.getElementById("id1").inne
rHTML = "Welcome to ExcelR";

   <script>
<body>
```

- "Accessing the paragraph and add content using innerHTML as follows":

```
<body>
      <p id="id2"></p>

   <script>

       document.getElementById("id2").inn
erHTML = "Hello All.. This is Java Full
Stack Class...!!!"

   <script>
<body>
```

2)getElementsByTagName():

getElementsByTagName() is a method in JavaScript used for DOM (Document Object Model) manipulations. It allows you to select elements by their tag name (e.g., div, p, span). Unlike getElementById(), which returns a single element, getElementsByTagName() returns a live HTMLCollection of all elements with the specified tag name.

Syntax:

```
document.getElementsByTagName('tagName')
```

```
  <body>
        <h1></h1>
        <h1></h1>
        <h2></h2>
        <h2></h2>
        <p></p>
        <p></p>
        <h1></h1>
     <script>
   let all_h1 =
document.getElementsByTagName("h1");

        for(let h1 of all_h1){
            h1.innerHTML = "First h1,
second h1, third h1";
        }
      </script>
</body>
```

- Accessing all “h2” elements

```
<script>
let all_h2 =
document.getElementsByTagName("h2");

        for(let h2 of all_h2){
            h2.innerHTML = "First h2,
second h2, third h2";
        }
</script>
```

- Accessing all paragraphs

```
<script>
let all_p =
document.getElementsByTagName("p");

        for(let p of all_p){
            p.innerHTML = "First p,
second p, third p";
        }
</script>
```

3)getElementsByClassName():

getElementsByClassName() is a method in JavaScript used for DOM (Document Object Model) manipulations. It allows you to select all elements that have a specific class name. Like getElementsByTagName(), this method returns a live HTMLCollection of elements that match the specified class name.

Syntax:

```
document.getElementsByClassName('className');
```

```
   <body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
      let all_p =
document.getElementsByClassName("c1");

        for(let p of all_p) {
            p.innerHTML = "Hello";
            p.style.color = "red";
        }
   </script>
 <body>
```

- Accessing all paragraphs with the class "C2".

```
<body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
      let all_p =
document.getElementsByClassName("c2");

        for(let p of all_p) {
            p.innerHTML = "Welcome";
            p.style.color = "green";
        }
   </script>

 <body>
```

4) querySelector(): querySelector() is a powerful and versatile method in JavaScript used for DOM (Document Object Model) manipulations. It allows you to select the first element that matches a specified CSS selector within the document.

Syntax:

```
document.querySelector('selector');
```

```
<body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
      let p =
document.querySelector(".c1");

        p.innerHTML = "c1...I am
getting by querySelector()";
   </script>

 <body>
```

- Accessing first paragraph with the class "C2".

```
<body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
let p = document.querySelector(".c2");
        p.innerHTML = "c2...I am
getting by querySelector()";
   </script>

 <body>
```

5) querySelectorAll(): querySelectorAll() is a method in JavaScript used for DOM (Document Object Model) manipulations. It allows you to select all elements that match a specified CSS selector and returns a static NodeList of those elements. This method is similar to querySelector(), but instead of returning just the first match, it returns all matches.

Syntax:

```
document.querySelectorAll('selector');
```

```
<body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
let all_p =
document.querySelectorAll(".c1");
        for(let p of all_p) {
            p.innerHTML = "Hello";
            p.style.color = "red";
        }
   </script>
 <body>
```

- Accessing all paragraphs with the class "C2".

```
<body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
let all_p =
document.querySelectorAll(".c2");
        for(let p of all_p) {
            p.innerHTML = "Welcome";
            p.style.color = "green";
        }
   </script>
 <body>
```

Element Creation

1) createElement(): In DOM (Document Object Model) manipulations, createElement() is a method provided by the document object in JavaScript. It is used to create a new HTML element dynamically. This method doesn't add the newly created element to the DOM tree; it just creates it in memory. You can later add this element to the DOM by using methods like appendChild(), insertBefore(), or similar.

Syntax:

```
let newElement =
document.createElement('tagName');
```

2) createTextNode(): createTextNode() is a method in the DOM (Document Object Model) used to create a new text node in JavaScript. Unlike createElement(), which creates HTML elements, createTextNode() specifically creates a text node containing the text you specify.

Syntax:

```
let textNode =
document.createTextNode('text content');
```

3) createDocumentFragment(): createDocumentFragment() is a method in the DOM (Document Object Model) that creates a minimal, lightweight container known as a document fragment. A document fragment is a special type of node that serves as a temporary storage container for other DOM nodes.

Syntax:

```
let fragment =
document.createDocumentFragment();
```

Element Manipulations

1) appendChild(): appendChild() is a method in the DOM (Document Object Model) used to add a node as the last child of a specified parent node. This method is often used to insert new elements or nodes into the DOM, whether they were created dynamically with methods like createElement() or createTextNode() or if they were already existing nodes that need to be moved or restructured.

Syntax:

```
parentNode.appendChild(childNode);
```

```
<body>
        <div id="id1"></div>

<script>
    let h1 =
document.createElement("h1");
        h1.textContent = "ExcelR";
        document.getElementById("id1").
appendChild(h1);

</script>

 <body>
```

Q) What are the differences between innerHTML and textContent?

a) innerHTML and textContent are two properties in the DOM (Document Object Model) used to manipulate and retrieve content within HTML elements, but they function quite differently and have distinct use cases.

- innerHTML: innerHTML is used to get or set the HTML content of an element, including any HTML tags and nested elements.
- textContent: textContent is used to get or set the textual content of an element, ignoring any HTML tags.

2) removeChild(): removeChild() is a method in the DOM (Document Object Model) used to remove a specified child node from its parent node. This method is commonly used when you want to delete an element from the DOM, either to clean up the document or to dynamically modify the structure of a webpage.

Syntax:

```
parentNode.removeChild(childNode);
```

```
<body>
        <div id="id1">

        <h1 id="my_h1">Hello</h1>

        <h2 id="my_h2">Java</h2>
        </div>
```

```
<script>

let h2 =
document.getElementById("my_h2");
let div =
document.getElementById("id1");
div.removeChild(h2);
</script>

</body>
```

3) replaceChild():replaceChild() is a method in the DOM (Document Object Model) used to replace an existing child node of a parent node with a new node. This method is useful when you want to swap out one element for another in the DOM, effectively removing the old node and inserting the new one in its place.

Syntax:

```
parentNode.replaceChild(newChild,  oldChild);
```

```
<body>
        <div id="id1">

          <h2 id="my_h2">ExcelR</h2>
        </div>
<script>

let p = document.createElement("p");
        p.innerHTML = "Welcome";
        let div =
document.getElementById("id1");
        let h2 =
document.getElementById("my_h2");
        div.replaceChild(p,h2);
</script>

</body>
```

4)insertBefore(): insertBefore() is a method in the DOM (Document Object Model) used to insert a new node into the DOM tree before a specified existing child node of a parent node. This method allows you to insert new elements or nodes at a specific position within the children of a parent element, rather than simply appending them to the end.

Syntax:

```
parentNode.insertBefore(newNode, referenceNode);
```

```
<body>
   <div id="id1">

       <p id="my_p">Hello, ExcelR</p>
   </div>
<script>

let h6 = document.createElement("h6");
   h6.innerHTML = "Hello, DOM !!!";
   let p =
document.getElementById("my_p");
        let div =
document.getElementById("id1");
        div.insertBefore(h6,p);
</script>
</body>
```

5) cloneNode(): cloneNode() is a method in the DOM (Document Object Model) that creates a copy of a specified node. This method can be used to duplicate an element and optionally, its entire subtree (i.e., all of its child nodes). The cloned node is not automatically inserted into the document; it needs to be appended to a parent node to become part of the DOM.

Syntax:

```
let clone = originalNode.cloneNode(true);
```

```
<body>
        <div id="id1">

            <h1 id="my_h1">Hello</h1>
        </div>
<script>

    let h1 =
document.getElementById("my_h1");
let xerox_h1 = h1.cloneNode(true);
let div =
document.getElementById("id1");
        div.appendChild(xerox_h1);
</script>

</body>
```

Attribute Manipulation

1) getAttribute(): getAttribute() is a method in the DOM (Document Object Model) that is used to retrieve the value of a specified attribute from an HTML element. This method is useful when you need to access custom data attributes, or standard attributes like id, class, href, src, etc., of an element.

Syntax:

```
let attributeValue = element.getAttribute(attributeName);
```

```
<body>
    <p id="my_p">Hello, ExcelR</p>

<script>

   let p =
document.getElementById("my_p");

document.write(p.getAttribute("id"));
</script>
</body>
```

2) setAttribute(): setAttribute() is a method in the DOM (Document Object Model) used to set or change the value of an attribute on an HTML element. This method allows you to add new attributes or update existing ones on an element.

Syntax:

```
element.setAttribute(attributeName, attributeValue);
```

```
<body>
    <p id="my_p">Hello, ExcelR</p>

<script>

  let p =
document.getElementById("my_p");

   p.setAttribute("key1","ExcelR");

document.write(p.getAttribute("key1"));
</script>

</body>
```

3)removeAttribute(): removeAttribute() is a method in the DOM (Document Object Model) used to remove an attribute from an HTML element. This method deletes the specified attribute from the element, effectively removing any associated value or behavior.

Syntax:

```
element.removeAttribute(attributeName);
```

```
<body>
    <p id="my_p">Hello, ExcelR</p>

<script>

p.setAttribute("key1","ExcelR");

document.write(p.getAttribute("id"));
document.write(p.getAttribute("key1"));
p.removeAttribute("key1");

</script>

</body>
```

4) hasAttribute(): hasAttribute() is a method in the DOM (Document Object Model) used to check whether a specified element has a particular attribute. This method is useful for determining if an element includes a specific attribute before attempting to work with it.

Syntax:

```
let hasAttr =
element.hasAttribute(attributeName);
```

```
<body>
    <p id="my_p">Hello, ExcelR</p>

<script>

   let p =
document.getElementById("my_p");

document.write(p.hasAttribute("id"));

</script>
</body>
```

CSS manipulation

1) classList.add():classList.add() is a method in the DOM (Document Object Model) used to add one or more class names to an element's list of classes. This method is a part of the classList property, which provides a convenient way to manipulate the class attribute of HTML elements.

Syntax:

```
element.classList.add(className1,
className2, ...);
```

```
<head>
        <title>DOM</title>
        <style>
            .c1{
                color: red;
            }
            .c2{
                color: green;
            }
        </style>
 </head>
<body>
<p id="my_p">Hello,ExcelR</p>
<button
onclick="func_one()">ClickMe</button>
<script>

let p =
document.getElementById("my_p");
  let func_one = ()=>{
    p.classList.add("c1");
      }
</script>

</body>
```

2) classList.remove(): classList.remove() is a method in the DOM (Document Object Model) used to remove one or more class names from an element's list of classes. It is part of the classList property, which provides an easy way to manage the class attribute of HTML elements.

Syntax:

```
element.classList.remove(className1,
className2, ...);
```

```
<head>
        <title>DOM</title>
        <style>
            .c1{
                color: red;
            }
            .c2{
                color: green;
            }
        </style>
 </head>
<body>
<p id="my_p">Hello,ExcelR</p>
<button
onclick="func_one()">ClickMe</button>
<button
onclick="func_two()">Remove</button>

<script>

let p = document.getElementById("my_p");
  let func_one = ()=>{
    p.classList.add("c1");
      }

let func_two = ()=>{
    p.classList.remove("c1");
     }
</script>
</body>
```

3) classList.contains(): classList.contains() is a method in the DOM (Document Object Model) used to check if an element has a specific class. This method is part of the classList property, which provides convenient access to an element's list of classes.

Syntax:

```
let hasClass =
element.classList.contains(className);
```

```
<head>
        <style>
            .c1{
                color: red;
            }
            .c2{
                color: green;
            }
        </style>
 </head>
```

```
<body>
<p id="my_p">Hello,ExcelR</p>
<button
onclick="func_one()">ClickMe</button>
<button
onclick="func_two()">Remove</button>
<button
onclick="func_three()">Check</button>


<script>

let p = document.getElementById("my_p");
  let func_one = ()=>{
    p.classList.add("c1");
      }

let func_two = ()=>{
    p.classList.remove("c1");
     }

let func_three= ()=>{
console.log(p.classList.contains("c1"));
      //true or false
            }
</script>
</body>
```

4) classList.toggle(): classList.toggle() is a method in the DOM (Document Object Model) used to add or remove a class from an element's list of classes, depending on whether the class is already present. This method is a part of the classList property, which simplifies the management of an element's class attribute. (Combination of classList.add() and classList.remove() called as “Toggle”).

Syntax:

```
element.classList.toggle(className, [force]);
```

```
<head>
        <style>
            .c1{
                color: red;
            }
            .c2{
                color: green;
            }
        </style>
 </head>
```

```
<body>
<p id="my_p">Hello,ExcelR</p>
<button
onclick="func_one()">ClickMe</button>
<button
onclick="func_two()">Remove</button>
<button
onclick="func_three()">Toggle</button>

<script>

let p = document.getElementById("my_p");
  let func_one = ()=>{
    p.classList.add("c1");
      }

let func_two = ()=>{
    p.classList.remove("c1");
     }

let func_three = ()=>{
    p.classList.toggle("c1");
       }
</script>
</body>
```

Event Handling

1) addEventListener(): addEventListener() is a method in the DOM (Document Object Model) used to attach event handlers to DOM elements. This method allows you to specify a function that will be executed when a particular event occurs on the element.

Syntax:

element.addEventListener(eventType, eventHandler);

```
<body>
<button id="my_btn">Button1</button>
<script>

let btn =
document.getElementById("my_btn");
btn.addEventListener("click",func_one);
function func_one(){
console.log("Hello");            //Hello
            }
</script>
</body>
```

2)removeEventListener():

removeEventListener() is a method in the DOM (Document Object Model) used to remove an event listener that was previously added to an element using addEventListener(). This method allows you to stop handling a particular event type on an element by removing the associated event handler.

Syntax:

element.removeEventListener(eventType, eventHandler);

```
<body>
<button id="my_btn2">Button2</button>
<script>

let btn2 =
document.getElementById("my_btn2");
btn2.addEventListener("click",func_two);
  setTimeout(function(){
btn2.removeEventListener("click",func_two
);
 },2000);
   function func_two(){
   console.log("Hello");        //Hello
       }
</script>
</body>
```

3) dispatchEvent(): dispatchEvent() is a method in the DOM (Document Object Model) used to programmatically trigger an event on a DOM element. This method allows you to create and dispatch custom events or simulate standard events, making it possible to programmatically interact with event-driven behaviors in your application.

Syntax:

element.dispatchEvent(event);

```
<body>
 <button id="my_btn3">Button3</button>
<script>
```

```
let btn3 =
document.getElementById("my_btn3");
function handleClick(event) {
alert('Button was clicked
programmatically!');
    }
  btn3.addEventListener('click',
handleClick);
  let clickEvent = new Event('click');
  btn3.dispatchEvent(clickEvent);
        </script>
    </body>
```

Q) Write a program so that whenever the login button is clicked, a success message is shown in green color if the login is successful; otherwise, a red-colored message is displayed, using DOM manipulation.

```
<!DOCTYPE html>
<html>
    <head>
        <title>Login</title>
        <style>
            #message {
                display: none;
                margin-top: 20px;
                font-size: 16px;
                padding: 10px;
                border-radius: 5px;
            }
            .success {
                color: white;
                background-color: green;
            }
            .failure {
                color: white;
                background-color: red;
            }

        </style>
    </head>
<body>
   <div>
<label for="username">Username:</label>
<input type="text" id="username">
   </div>
   <div>
<label for="password">Password:</label>
<input type="password" id="password">
   </div>
```

```
<button id="loginButton">Login</button>

<div id="message"></div>
<script>
document.getElementById('loginButton').
addEventListener('click', function() {

// Get the values of the username and
password inputs
 let username =
document.getElementById('username').val
ue;
 let password =
document.getElementById('password').val
ue;

// Get the message element
let msg =
document.getElementById('message');

// Check if the login is successful
(this is just a simple condition)
  if (username === 'ExcelR' && password
=== 'ExcelR@123') {
// Success: Show a success message in
green
msg.textContent = 'Login successful!';
msg.className = 'success';
} else {
// Failure: Show a failure message in
red
msg.textContent = 'Login failed. Please
try again.';
msg.className = 'failure';
}

// Display the message element
msg.style.display = 'inline-block';

});
        </script>
    </body>
</html>
```

Document object model:

Document object model used by javascript to define the structure to the element. In general it follows tree based or hierarchical based for easier and efficient access, store and retrieve elements,attributes or style of element and to do some manipulations. It represents the page as a tree of objects, where each node corresponds to a part of the document (such as an element, attribute, or text content).

Types of Dom manipulations

Element accessing	Attribute Manipulation	Element Manipulation	Event Handling	CSS manipulation	Element Creation
getElementById()	getAttribute()	appendChild()	addEventListener()	classList.add()	createElement
getElementsByTagName()	setAttribute()	removeChild()	removeEventListener()	classList.containes()	createTextNode()
getElementsByClassName()	removeAttribute()	replaceChild()	dispatchEvent()	classList.remove()	createDocumentFragment()
querySelector()	hasAttribute()	insertBefore()	—---------	classList.toggle()	—---------
querySelectorAll()	—----------	CloneNode()	—----------	—----------	—----------

Element accessing

1. **getElementById():** getElementById() is a widely used method in JavaScript for DOM (Document Object Model) manipulations. It allows you to select and manipulate HTML elements by their id attribute.

Here's how it works and how you can use it:

Syntax:

document.getElementById('id')

Ex:

```
<body>
<h1 className=”heading1tag">Component My Profile page</h1>
<script>
document.getElementById("id1").innerHTML = "Welcome ";<script>
```

<body>
"Accessing the paragraph and add content
using innerHTML as follows":
<body>
<p id="
id2"></p>
<script>
document.getElementById("id2").inn
erHTML = "Hello All.. This is Java Full
Stack Class...!!!"
<script>
<body>

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>DOM Manipulation Demo</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header>
        <h1>DOM Manipulation Demo</h1>
        <p>Visitor Count: <span id="visitorCount">0</span></p>
    </header>

    <section>
        <h2>Dynamic Content Display</h2>
        <button class="action-button" onclick="displayType('Text')">Show
Text</button>
        <button class="action-button" onclick="displayType('Image')">Show
Image</button>
        <button class="action-button"
onclick="displayType('Animation')">Show Animation</button>
        <div id="displayArea" class="display-area"></div>
    </section>

    <section>
```

```
        <h2>Image Toggle</h2>
        <img id="toggleImage"
src="https://www.kluniversity.in/fphotos/actual/7633.jpg"
alt="Placeholder" />
        <img id="toggleImage"
src="https://www.kluniversity.in/fphotos/actual/6655.jpg" alt="faculty"/>
        <img id="toggleImage"
src="https://www.kluniversity.in/fphotos/actual/4592.jpg" alt="faculty2"/>
        <button class="action-button" id="toggleImageButton">Toggle
Image</button>

    </section>

    <script src="script.js"></script>
</body>
</html>
```

----------script.js----------

```
// Increment Counter
const countElement = document.getElementById('count');
const incrementButton = document.getElementById('incrementButton');
let count = 0;

incrementButton.addEventListener('click', () => {
    count++;
    countElement.textContent = count;
    document.title = `Count: ${count}`; // Update page title dynamically
});

// Form Handling
const textInput = document.getElementById('textInput');
const outputText = document.getElementById('outputText');

textInput.addEventListener('input', () => {
    outputText.textContent = textInput.value;
});

// Toggle Image
```

```
const dynamicImage = document.getElementById('dynamicImage');
const toggleImageButton = document.getElementById('toggleImageButton');
const placeholder1 = "https://via.placeholder.com/150";
const placeholder2 = "https://via.placeholder.com/300";

toggleImageButton.addEventListener('click', () => {
   dynamicImage.src = dynamicImage.src === placeholder1 ? placeholder2 :
placeholder1;
});

// Scroll to Section
const scrollButton = document.getElementById('scrollButton');
const animatedSection = document.getElementById('animatedSection');

scrollButton.addEventListener('click', () => {
   animatedSection.scrollIntoView({ behavior: 'smooth' });
});

// Element Visibility Detection
const visibilityBox = document.getElementById('visibilityBox');

const observer = new IntersectionObserver(
   ([entry]) => {
       visibilityBox.textContent = entry.isIntersecting ? "Visible" : "Not
Visible";
       visibilityBox.style.background = entry.isIntersecting ? "green" :
"red";
   },
   { threshold: 0.1 }
);

observer.observe(visibilityBox);
```

----------styles.css----------

```css
body {
    font-family: Arial, sans-serif;
    line-height: 1.5;
    padding: 20px;
}

header {
    background: #282c34;
    color: white;
    padding: 20px;
    text-align: center;
    margin-bottom: 20px;
}

.action-button {
    padding: 10px 20px;
    margin: 10px 5px;
    cursor: pointer;
    background: #007bff;
    color: white;
    border: none;
    border-radius: 5px;
}

.action-button:hover {
    background: #0056b3;
}

.display-area {
    margin-top: 20px;
    padding: 20px;
    border: 1px solid #ccc;
    text-align: center;
    min-height: 150px;
}

img {
    max-width: 100%;
```

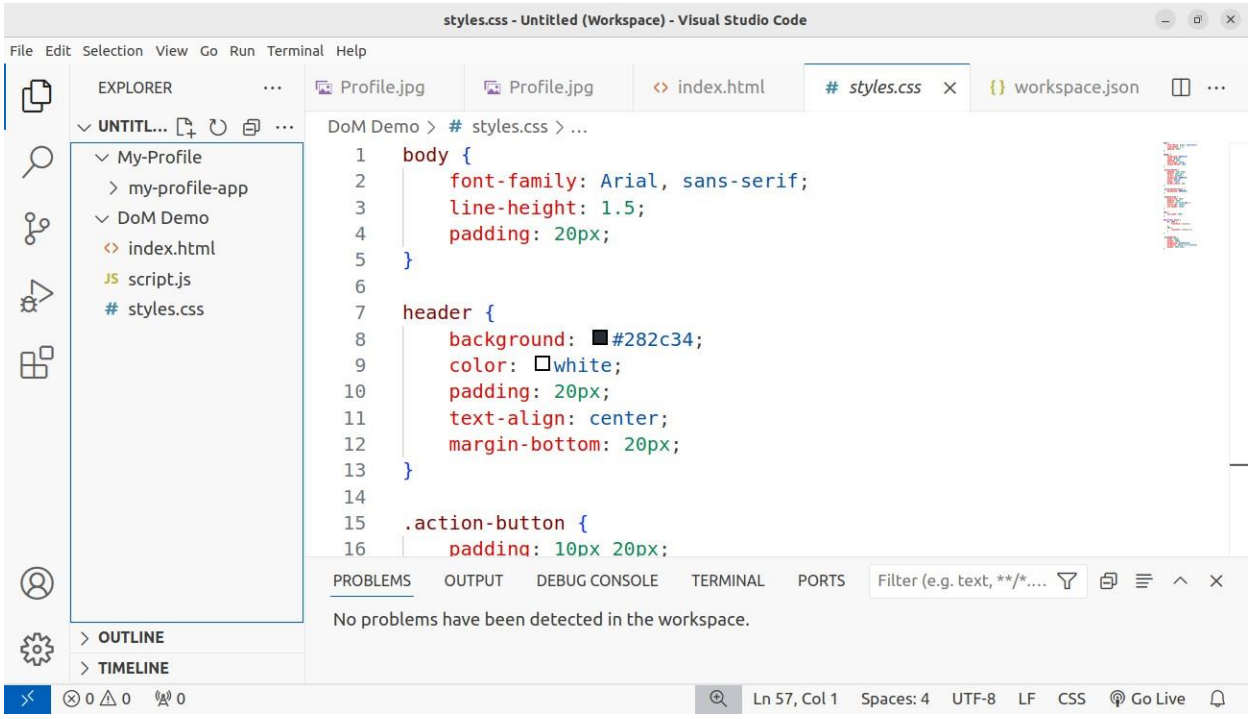
```
}

@keyframes pulse {
   0%, 100% {
       transform: scale(1);
   }
   50% {
       transform: scale(1.2);
   }
}

.animated-box {
   width: 100px;
   height: 100px;
   background: lightgreen;
   animation: pulse 1.5s infinite;
   margin: 20px auto;
}
```

Save the files in same folder DOM Demo : index.html script.js and styles.css
Press go live at bottom

Output from Browser:


styles.css - Untitled (Workspace) - Visual Studio Code
File Edit Selection View Go Run Terminal Help
EXPLORER
UNTITL...
My-Profile
my-profile-app
DoM Demo
index.html
script.js
styles.css
OUTLINE
TIMELINE
Profile.jpg
Profile.jpg
index.html
styles.css
workspace.json
DoM Demo > # styles.css > ...
1 body {
2 font-family: Arial, sans-serif;
3 line-height: 1.5;
4 padding: 20px;
5 }
6
7 header {
8 background: #282c34;
9 color: white;
10 padding: 20px;
11 text-align: center;
12 margin-bottom: 20px;
13 }
14
15 .action-button {
16 padding: 10px 20px;
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Filter (e.g. text, **/*....
No problems have been detected in the workspace.
0 0 0
Ln 57, Col 1 Spaces: 4 UTF-8 LF CSS Go Live

PROMISES

- Promises are special objects in java
- promises are used to make server calls, API calls, and handle asynchronous operations.
- Promises are a modern alternative to callbacks for handling asynchronous operations in JavaScript.
- A Promise in JavaScript can have three states:
  - **Pending**: The initial state of a promise neither fulfilled nor rejected.
  - **Fulfilled**: The state of a promise when the operation is completed successfully.
  - **Rejected**: The state of a promise when the operation fails.
- We create a promise by instantiating the 'Promise' class.
- We can consume a promise in two ways:
  1) Using the 'then' method.
  2) Using 'async' and 'await' (introduced in ES6).

EXAMPLE-1

```
<script>
letPromise1=new Promise((resolve,reject)=>{
            resolve("welcome!!")
        })
      //consume by using then
           Promise1.then((posres)=> {
           document. Write(posres)
        }, (Negres)=> {
            document. Write (Negres)
        })
    Output: welcome!!
     //consume by using async and await

        let consume=async () => {
         let res = await Promise1
         document. Write(res)
        }
        consume ()
    Output: welcome!!
</script>
```

EX-2: Handle posRes after 5 Seconds

```
<script>
  //consume by using async and await
letPromise1=new Promise ((resolve, reject)=>{
        setTimeout(()=>{
                resolve("success!!")
            },5000)
        })
        let consume=async ()=>{
              let res=await Promise1
              document. Write(res)
        }
        consume ()
    Output: welcome!! (After 5 seconds)
```

EX-3: Handle posRes after 0sec,5sec,10sec

```
<script>
let Promise1=new Promise((resolve,reject)=>{
         setTimeout (() => {
             resolve("welcome")
         },0)
     })
let Promise2=new Promise((resolve,reject)=>{
         setTimeout (() => {
             resolve("to")
             // reject("error!!")
         },5000)
     })
let Promise3=new Promise((resolve,reject)=>{
         setTimeout (() => {
             resolve("excelr")
         },10000)
     })
     let consume=async ()=>{
         let res1=await Promise1
         document. Write(res1)
         let res2=await Promise2
         document. Write(res2)
         let res3=await Promise3
         document. Write(res3)
     }
     consume ()
  Output:welcometoexcelr

</script>
```

Promise. All()

- The Promise.all function is used to consume multiple promises simultaneously, returning a single promise that resolves when all of the promises in the array have resolved.
- The Promise.all function returns an array containing the resolved values of all the promises passed to it.
- The result of the promises will be visible after the maximum time taken by any of the promises to resolve.

EX-1

```
<script>
let Promise1=new Promise((resolve,reject)=>{
        setTimeout(()=>{
            resolve("hello")
        },0)
     })
let Promise2=new Promise((resolve,reject)=>{
        setTimeout(()=>{
            resolve("sapuru")

        },5000)
     })
let Promise3=new Promise((resolve,reject)=>{
        setTimeout(()=>{
            resolve("dinesh")
        },10000)
     })
let consume=async()=>{
let res= await
Promise.all([Promise1,Promise2,Promise3])
        console.log(res)
    }
    consume()
<script>
Output: ['welcome', 'to', 'excelr']
```

AllSettled()

- The `Promise.allSettled` function returns a consolidated result of all promises, providing an array of objects that describe the outcome of each promise, whether it was fulfilled or rejected.
- A consolidated result contains both fulfilled and rejected promises.

EXAMPLE

```
<script>
let Promise1=new Promise((resolve,reject)=>{
        setTimeout(()=>{
            resolve("hello")
        },0)
     })
let Promise2=new Promise((resolve,reject)=>{
        setTimeout(()=>{
            resolve("sapuru")
            // reject("error!!")
        },5000)
     })
let Promise3=new Promise((resolve,reject)=>{
        setTimeout(()=>{
            resolve("dinesh")
        },10000)
     })
let consume=async()=>{
let res= await
Promise.all([Promise1,Promise2,Promise3])
        console.log(res)
    }
    consume ()
<script>
Output:
0: {status: 'fulfilled', value: 'welcome’}
1: {status: 'fulfilled', value: 'to'}
2: {status: 'fulfilled', value: 'excelr'}
```

Race ()

- The `Promise.race` function in JavaScript is used to run multiple promises concurrently and returns a single promise that resolves or rejects as soon as the first promise in the array resolves or rejects.
- The `race` function is used to determine which promise resolves or rejects first.

Example:

```
<script>
let Promise1=new Promise((resolve,reject)=>{
        setTimeout(()=>{
            resolve("welcome")
        },0)
     })
let Promise2=new Promise((resolve,reject)=>{
        setTimeout(()=>{
            resolve("to")
            // reject("error!!")
        },5000)
     })
let Promise3=new Promise((resolve,reject)=>{
        setTimeout(()=>{
            resolve("excelr")
        },10000)
     })
let consume=async ()=>{
let res=await
Promise.race([Promise1,Promise2,Promise3])
       console.log(res)
    }
    consume()
</script>
Output: welcome
```

Interview question

```
<script>
let Promise1=new Promise((resolve,reject)=>{
        setTimeout(()=>{
            resolve("hello")
        },0)
     })
let Promise2=new Promise((resolve,reject)=>{
            resolve("welcome")
     })
let consume=async ()=>{
let res=await
Promise.race([Promise1,Promise2])
        console.log(res)
     }
     consume()
</script>
output: welcome
        hello
```

In this lecture , We discuss about Different types of Components type adding to React Project: For that , we are using

1. **Type Rfc -type and press enter in vs code**

   You get syntax structure like this:

```
import React from 'react'

export default function Login() {
 return (
   <div>Login</div>
 )
}
```

Note:

When to use:

When you need a simple functional component with **no hooks or state.**

2. **Type rcc (React Class Component):**

```
import React, { Component } from 'react'

export default class Login extends Component {
 render() {
   return (
     <div>Login</div>
   )
 }
}
```

Note : **When to use:**

When you need lifecycle methods or state management within a class-based structure. (Less commonly used after React hooks were introduced.)

3. **Type rafc(react arrow function Components)**

```
import React from 'react'

export const Login = () => {
 return (
   <div>Login</div>
 )
}
```

Note: When you want to create a reusable functional component and use named exports.

4. **Type rafce -(React arrow Function with Component Export)**

```
import React from 'react'

const Login = () => {
 return (
   <div>Login</div>
 )
}

export default Login
```

Note : When you need an arrow function component and prefer default exports.

These shortcuts make it quick and easy to scaffold components in a React project, saving time on boilerplate code. Be sure to install the **React ES7+ Snippets** extension in VS Code to enable these snippets!

Virtual DOM Demonstration- React:

Pre-Requirements:

Fontawesome icons

npm install --save @fortawesome/react -fontawesome @fortawesome/free-brands-svg-icons @fortawesome/fontawesome-svg-core

Redux - React:

Aim: To Implement Vite app with JavaScript and Redux for a single-page job portal application.

Procedure:

create a single store, a reducer, and use useSelector and useDispatch for managing state.

Steps:

Create a Vite App in VS Code

```
npm create vite@latest job-portal --template react
cd job-portal
npm install
npm install @reduxjs/toolkit react-redux
```

Install Redux Toolkit and React-Redux:

```
npm install @reduxjs/toolkit react-redux
```

Project File Structure:

```
src/
├── app/
│   └── store.js
├── features/
│   └── jobs/
│       ├── jobsSlice.js
├── components/
│   ├── JobList.jsx
│   ├── AddJob.jsx
└── App.jsx
```

Now , Create folder named app under src and inside create a file , store.js, and paste code.--------- >This file sets up the Redux store.

Then src/features/jobs/jobsSlice.js features folder under src, create sub-folder jobs under new file name its as jobsSlice.js and paste code.----------> This slice manages the job-related state.

Again under src folder components , new file named JobList.jsx and Paste code .
Repeat paste code in appropriate paths as follows:
src/components/JobList.jsx---------- >This component displays the list of jobs.
src/components/AddJob.jsx --------- > This component allows adding a new job.
src/App.jsx ----------> This file integrates the components.

Code Implementation:

src/app/store.js

```
import { configureStore } from '@reduxjs/toolkit';
import jobsReducer from '../features/jobs/jobsSlice';
export const store = configureStore({
  reducer: {
    jobs: jobsReducer,
  },
});
```

src/features/jobs/jobsSlice.js

```
import { createSlice } from '@reduxjs/toolkit';
const initialState = {
  jobs: [],
};
const jobsSlice = createSlice({
  name: 'jobs',
  initialState,
  reducers: {
    addJob: (state, action) => {
      state.jobs.push(action.payload);
    },
    removeJob: (state, action) => {
      state.jobs = state.jobs.filter(job => job.id !== action.payload);
    },
  },
}); export const { addJob, removeJob } = jobsSlice.actions;
export default jobsSlice.reducer;
```

src/components/JobList.jsx

```
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { removeJob } from '../features/jobs/jobsSlice';

const JobList = () => {
  const jobs = useSelector(state => state.jobs.jobs);
  const dispatch = useDispatch();

  return (
    <div>
      <h2>Job List</h2>
      {jobs.length === 0 && <p>No jobs available</p>}
      <ul>
        {jobs.map(job => (
          <li key={job.id}>
            <span>{job.title}</span>
            <button onClick={() => dispatch(removeJob(job.id))}>Delete</button>
          </li>
        ))}
      </ul>
    </div>
  );
}; export default JobList;
```

src/components/AddJob.jsx

```
import React, { useState } from 'react';
import { useDispatch } from 'react-redux';
import { addJob } from '../features/jobs/jobsSlice';

const AddJob = () => {
  const [jobTitle, setJobTitle] = useState('');
  const dispatch = useDispatch();

  const handleAddJob = () => {
    if (jobTitle.trim()) {
      dispatch(addJob({ id: Date.now(), title: jobTitle }));
      setJobTitle('');
    }
  };

  return (
    <div>
      <h2>Add Job</h2>
      <input
```

```
        type="text"
        value={jobTitle}
        onChange={e => setJobTitle(e.target.value)}
        placeholder="Job Title"
      />
      <button onClick={handleAddJob}>Add</button>
    </div>
  );
};

export default AddJob;
```

src/App.jsx

This file integrates the components.

```
import React from 'react';
import { Provider } from 'react-redux';
import { store } from './app/store';
import JobList from './components/JobList';
import AddJob from './components/AddJob';

const App = () => {
  return (
    <Provider store={store}>
      <div>
        <h1>Job Portal</h1>
        <AddJob />
        <JobList />
      </div>
    </Provider>
  );
};

export default App;
```

Execution :

Finally, Run the App: use command in VS code terminal

npm run dev.

Features:

Add Job: Users can add a new job by entering its title.
Delete Job: Users can delete a job from the list.
State Management: Redux is used to manage the state of the job list.

This structure is scalable and can be extended for more complex features, such as job filtering, search, or detailed job descriptions.

TASK 2: integrate projects and Expected outputs :

HomeAbout

About Job Portal

Job Portal use Redux - Global State Management.

Job Portal

Add Job

Job Title Add

Job List

No jobs available

HomeAbout

Welcome to Home Page

Next Prev

Job Portal

Add Job

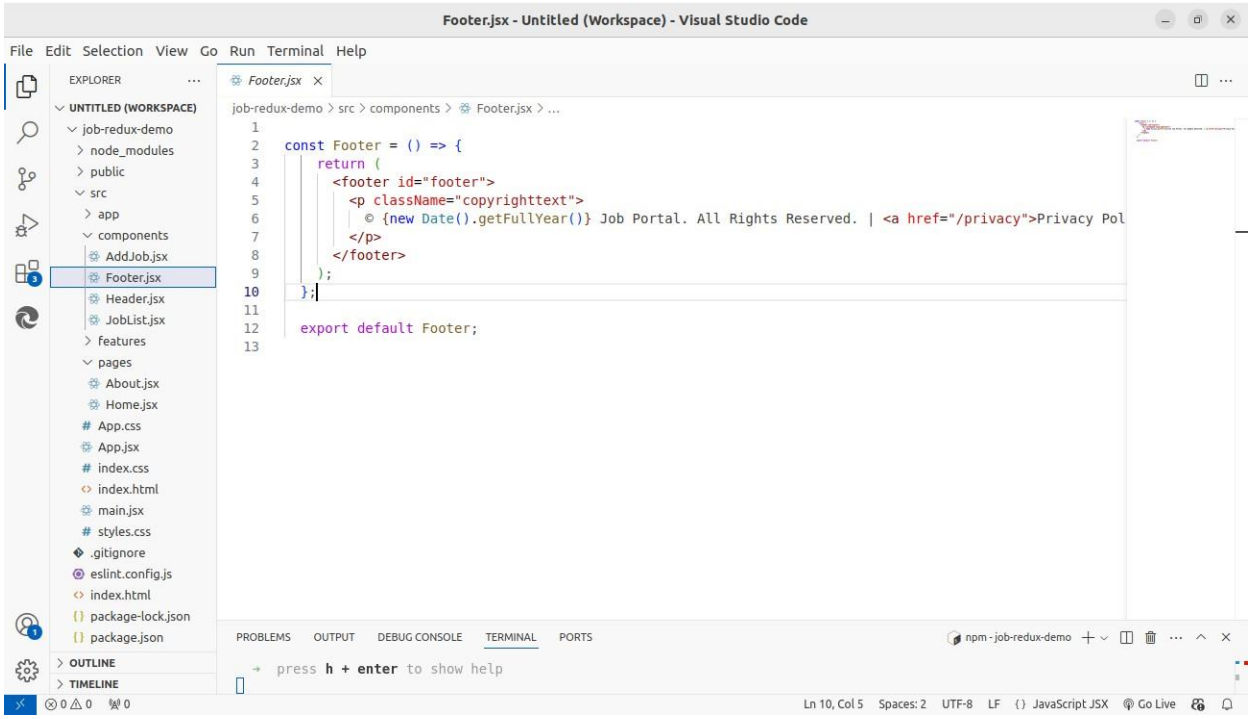
Job Title Add

Job List

Job ID-23142025-AI Specialist Delete

Job ID-23101025-SOFTWARE TESTER- LOCATION: MUMBAI Delete

FRESHER- DevOps -Bangalore Delete



Later Footer.jsx code included in app.js

```
const Footer = () => {
   return (
     <footer id="footer">
       <p className="copyrighttext">
         © {new Date().getFullYear()} Job Portal. All Rights Reserved. |
<a href="/privacy">Privacy Policy</a>
       </p>
     </footer>
   );
 };
  export default Footer;
```

Header .jsx code included in app.js

```
import { Link } from "react-router-dom";

const Header = () => {
 return (
   <header id="header">
     <div className="logo">
       <img src="/logo.png" height={40} width={40} alt="Logo" />
       <span className="logotext">Job Portal</span>
     </div>
     <nav>
       <Link to="/">Home</Link>
       <Link to="/about">About</Link>
     </nav>
   </header>
 );
};

export default Header;
```

Under pages folder: Home.jsx and About.jsx

```
import React from "react";
import "../styles.css";

const Home = () => {
 return (
   <>
     <div id="header">
       <h1> Welcome to Home Page</h1>
       <div
         style={{
           width: 100,
           height: 70,
           background: "aquagreen",
         }}
       />
     </div>


      <div>

       <button>Next</button>
       <button>Prev</button>
     </div>
   </>
 );
};

export default Home;
```

Under src styles.css

```
      /* Global reset and layout setup */
* {
 margin: 0;
 padding: 0;
```

```
 box-sizing: border-box;
}

html, body {
 height: 100%;
 font-family: Arial, sans-serif;
}

/* Main container */
#container {
 display: grid;
 grid-template-columns: 1fr;    /* Single column layout */
 grid-template-rows: auto 1fr auto;  /* Header, Content, Footer */
 grid-template-areas:
   "header"
   "content"
   "footer";
 height: 100vh;  /* Take up the full viewport height */
}

/* Header section */
#header {
 grid-area: header;
 background: yellow;
 padding: 20px;
 display: flex;
 justify-content: space-between;
 align-items: center;
}

#header .logo {
 display: flex;
 align-items: center;
}

#header .logo img {
 margin-right: 10px;
}

#header nav a {
```

```
 margin-left: 20px;
 text-decoration: none;
 color: black;
}

#header nav a:hover {
 text-decoration: underline;
}

/* Main content section */
#content {
 grid-area: content;
 padding: 20px;
 background: bisque;
 overflow-y: auto;  /* Allow scrolling if content exceeds the viewport */
}

#footer {
 grid-area: footer;
 background: pink;
 text-align: center;
 padding: 10px;
}

#footer .copyrighttext {
 font-size: 12px;
 color: gray;
}
```

Index.css

```
html,body{
   margin: 0px;
   width: 100%;
   height: 100%;
   font-family: 'Times New Roman', Times, serif;
   font-size: 10pt;
}
#root{
   width: 100%;
```

```
   height: 100%;
}
:focus{
   outline: none;
}
```

App.jsx

```
import { Routes, Route, Link } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";
import { Provider } from 'react-redux';
import { store } from './app/store';
import JobList from './components/JobList';
import AddJob from './components/AddJob';

const App = () => {
 return (
   <Provider store={store}>
     <div>
       <Header />
       <main>
         <Routes>
           <Route path="/" element={<Home />} />
           <Route path="/about" element={<About />} />
         </Routes>
         <h1>Job Portal</h1>
         <AddJob />
         <JobList />
       </main>
       <Footer />
     </div>
   </Provider>
 );
};

const Header = () => {
 return (
   <header>
     <div>
```

```
        <img src="/logo.png" height={90} width={75} alt="Job Portal Logo"
/>
      </div>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>
    </header>
  );
};

const Footer = () => {
  return (
    <footer>
      <p>
        © {new Date().getFullYear()} KLEF Job Portal. All Rights Reserved.
| <a href="/privacy">Privacy Policy</a>
      </p>
    </footer>
  );
};

export default App;
```

Backend Development with Java Spring Framework

Spring Boot Fundamentals

Architecture

- **MVC Pattern**:
  - **Model**: Data layer (e.g., entities like `User`).
  - **View**: Frontend (React in this course).
  - **Controller**: Handles HTTP requests.

```
@RestController @RequestMapping("/api") public class HelloController {
@GetMapping("/hello")   public String sayHello() {     return "Hello, World!";
  }
}
```

- **Dependency Injection**:
  - **Concept**: Beans are managed by Spring's IoC container.
  - **Example**:

```
@Service public class UserService {
private final UserRepository repo;
  @Autowired   public
UserService(UserRepository repo) {
this.repo = repo;
  }   public List<User>
findAll() {     return
repo.findAll();
  }
}
```

Setting Up a Spring Boot Project

- **Spring Initializr**:
  - URL: start.spring.io (https://start.spring.io)
  - Dependencies: Spring Web, Spring Data JPA, MySQL Driver, Lombok.
  - Generate and import into STS.
- **Project Structure**:

```
myapp/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/example/myapp/ │   │
│       ├── MyappApplication.java
│   │   │       ├── controller/ │
│   │       ├── service/
│   │   │       ├── repository/
│   │   │       └── model/
│   │   └── resources/
│   │       └── application.properties
└── pom.xml
```

- **pom.xml**:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
  </dependency>
```

```
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
  </dependency>
</dependencies>
```

RESTful APIs

- **Controller**:

```
@RestController
@RequestMapping("/api/users") public
class UserController {
  @Autowired   private UserService
userService;
  @GetMapping   public List<User>
getUsers() {     return
userService.findAll();
  }
  @GetMapping("/{id}")   public User getUser(@PathVariable Long id) {     return
userService.findById(id).orElseThrow(() -> new ResourceNotFoundException("User not found"));
  }
  @PostMapping   public User
createUser(@RequestBody User user) {     return
userService.save(user);
  }
  @PutMapping("/{id}")   public User updateUser(@PathVariable Long id,
@RequestBody User user) {     user.setId(id);     return
userService.save(user);
  }
  @DeleteMapping("/{id}")   public void
deleteUser(@PathVariable Long id) {
userService.delete(id);
  }
}
```

- **HTTP Status Codes**:
  - 200: OK
  - 201: Created
  - 404: Not Found
  - 400: Bad Request

Data Access with JPA

- **Entity**:

```
@Entity
@Table(name = "users")
@Data public class
User {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
  @Column(nullable = false)
private String name;
@Column(unique = true)   private
String email;
}
```

- **Repository**:

```
public interface UserRepository extends JpaRepository<User, Long> {
  Optional<User> findByEmail(String email);
}
```

- **Service**:

```
@Service public class
UserService {
  @Autowired   private
UserRepository repo;   public
List<User> findAll() {
return repo.findAll();
  }   public Optional<User> findById(Long
id) {     return repo.findById(id);
  }   public User save(User
user) {     return
repo.save(user);
  }   public void delete(Long
id) {     repo.deleteById(id);
  }
}
```

Database Configuration

- **application.properties**:

```
spring.datasource.url=jdbc:mysql://localhost:3306/mydb?useSSL=false&serverTimezone=UTC
spring.datasource.username=root spring.datasource.password=secret123
spring.jpa.hibernate.ddl-auto=update spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

- **Schema Setup**:

```
CREATE DATABASE mydb;
USE mydb;
```

Error Handling and Validation

- **Exception Handling**:

```
public class ResourceNotFoundException extends RuntimeException {
public ResourceNotFoundException(String message) {
super(message);
  }
}
@ControllerAdvice public class GlobalExceptionHandler {
@ExceptionHandler(ResourceNotFoundException.class)   public
ResponseEntity<String> handleNotFound(ResourceNotFoundException ex) {
return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);   }
  @ExceptionHandler(Exception.class)   public ResponseEntity<String> handleGeneral(Exception ex) {
return new ResponseEntity<>("Server error: " + ex.getMessage(), HttpStatus.INTERNAL_SERVER_ERROR);
  }
}
```

- **Validation**:

```
@Entity public
class User {
  @NotBlank(message = "Name is required")
private String name;
  @Email(message = "Invalid email format")
@NotNull(message = "Email is required")   private
String email;
}
@RestController public class
UserController {
  @PostMapping   public ResponseEntity<User> createUser(@Valid @RequestBody User user,
BindingResult result) {     if (result.hasErrors()) {
      String errors = result.getAllErrors().stream()
        .map(ObjectError::getDefaultMessage)
.collect(Collectors.joining(", "));       return new ResponseEntity<>(null,
HttpStatus.BAD_REQUEST);     }     return new
ResponseEntity<>(userService.save(user), HttpStatus.CREATED);
  }
}
```

Integrating with React

- **CORS**:

```
@Configuration public class WebConfig implements
WebMvcConfigurer {
  @Override   public void addCorsMappings(CorsRegistry
registry) {     registry.addMapping("/api/**")
      .allowedOrigins("http://localhost:3000")
.allowedMethods("GET", "POST", "PUT", "DELETE");
  }
}
```

- **React Fetch**:

```
function UserList() {   const [users, setUsers]
= useState([]);   useEffect(() => {
axios.get("http://localhost:8080/api/users")
      .then(res => setUsers(res.data))
      .catch(err => console.error(err));
  }, []);   return <ul>{users.map(user => <li
key={user.id}>{user.name}</li>)}</ul>; }
```

Advanced Spring Boot Features

Spring Data JPA

- **Custom Queries**:

```
public interface UserRepository extends JpaRepository<User, Long> {
  List<User> findByNameContainingIgnoreCase(String name);
  @Query("SELECT u FROM User u WHERE u.email LIKE %:domain")
  List<User> findByEmailDomain(@Param("domain") String domain); }
```

- **Pagination**:

```
@GetMapping public Page<User> getUsers(@RequestParam int page, @RequestParam int
size) {   Pageable pageable = PageRequest.of(page, size,
Sort.by("name").ascending());   return userService.findAll(pageable);
}
```

Spring Boot with Other Databases

- **PostgreSQL**:
  - Dependency:

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>
```

  - Config:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

- **MongoDB**:
  - Dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
```

  - Entity:

```
@Document(collection = "users") public
class User {
  @Id   private String
id;   private String
name;   private String
email;
}
```

  - Config:

```
spring.data.mongodb.uri=mongodb://localhost:27017/mydb
```

API Documentation with Swagger

- **Setup**:
  - Dependencies:

```
<dependency>
  <groupId>io.springfox</groupId>
  <artifactId>springfox-swagger2</artifactId>
  <version>3.0.0</version>
</dependency>
<dependency>
  <groupId>io.springfox</groupId>
  <artifactId>springfox-swagger-ui</artifactId>
  <version>3.0.0</version>
</dependency>
```

  - Config:

```
@Configuration @EnableSwagger2 public class
SwaggerConfig {   @Bean   public Docket api() {
return new Docket(DocumentationType.SWAGGER_2)
      .select()
      .apis(RequestHandlerSelectors.basePackage("com.example.controller"))
      .paths(PathSelectors.any())
      .build()       .apiInfo(new
ApiInfoBuilder()
        .title("My API")
        .description("Full Stack App API")
        .version("1.0")
        .build());
  }
}
```

- Access: `http://localhost:8080/swagger-ui/`

Exercises

1. **Blog API**:
   - Entities: `Post`, `Comment`.
   - CRUD endpoints for both.
   - Validation: Title required, content min 10 chars.
   - Swagger documentation.
   - Test with Postman or curl.

Access Control and Advanced Backend Features

HTTP Handling in Depth

- **Request Lifecycle**:
  - Client sends request → Filters → Interceptors → Controller → Service → Repository → Response.
- **Filters**:

```
@Component public class LoggingFilter
implements Filter {
  @Override   public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain)       throws IOException, ServletException {
    HttpServletRequest request = (HttpServletRequest) req;
    System.out.println("Request: " + request.getMethod() + " " + request.getRequestURI());
chain.doFilter(req, res);
  }
}
```

- **Interceptors**:

```
@Component public class RequestInterceptor implements
HandlerInterceptor {
  @Override   public boolean preHandle(HttpServletRequest request, HttpServletResponse response,
{ Object handler) {     System.out.println("Pre-handle: " + request.getRequestURI());     return true;
  }
  @Override   public void postHandle(HttpServletRequest request, HttpServletResponse response, Object handler, ModelAndView
modelAndView)
    System.out.println("Post-handle");
  }
}
@Configuration public class WebConfig implements
WebMvcConfigurer {
  @Autowired   private
RequestInterceptor interceptor;
  @Override   public void
addInterceptors(InterceptorRegistry registry) {
registry.addInterceptor(interceptor);
  }
}
```

Security in Spring Boot

Authentication and Authorization

- **Spring Security Basics**:
  - Dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

  - Config:

```
@Configuration @EnableWebSecurity public class SecurityConfig
extends WebSecurityConfigurerAdapter {
  @Override   protected void configure(HttpSecurity http) throws
Exception {     http
      .authorizeRequests()
        .antMatchers("/public/**").permitAll()
        .anyRequest().authenticated()
      .and()
      .httpBasic();
  }   @Bean   public UserDetailsService
userDetailsService() {     UserDetails user =
User.withUsername("user")
      .password(passwordEncoder().encode("password"))
      .roles("USER")       .build();     return
new InMemoryUserDetailsManager(user);
  }   @Bean   public PasswordEncoder
passwordEncoder() {     return new
BCryptPasswordEncoder();
  }
}
```

- **JWT**:
  - Generate Token:

```
public String generateToken(UserDetails user) {
return Jwts.builder()
    .setSubject(user.getUsername())
    .setIssuedAt(new Date())
    .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60 * 10)) // 10
hours     .signWith(SignatureAlgorithm.HS512, "secretkey")     .compact();
}
```

- Filter:

```
@Component public class JwtFilter extends
OncePerRequestFilter {
  @Override   protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain chain)       throws ServletException, IOException {
    String header = request.getHeader("Authorization");
if (header != null && header.startsWith("Bearer ")) {
String token = header.substring(7);       try {
        Claims claims = Jwts.parser().setSigningKey("secretkey").parseClaimsJws(token).getBody();
        String username = claims.getSubject();
SecurityContextHolder.getContext().setAuthentication(           new
UsernamePasswordAuthenticationToken(username, null, new ArrayList<>())
        );
      } catch (Exception e) {
        SecurityContextHolder.clearContext();
      }     }
chain.doFilter(request, response);
  }
}
```

OAuth2

- **Setup**:
  - Dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>   <artifactId>spring-boot-starter-
oauth2-client</artifactId>
</dependency>
```

  - Config:

```
spring.security.oauth2.client.registration.google.client-id=your-client-id
spring.security.oauth2.client.registration.google.client-secret=your-client-secret
spring.security.oauth2.client.registration.google.scope=openid,email,profile
```

  - Security Config:

```
@Override protected void configure(HttpSecurity http) throws Exception {   http
    .authorizeRequests()
      .anyRequest().authenticated()
    .and()
    .oauth2Login();
}
```

Role-Based Access Control (RBAC)

- **User Entity**:

```
@Entity public
class User {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;   private String username;   private
String password;
  @ElementCollection(fetch = FetchType.EAGER)
private Set<String> roles = new HashSet<>();
}
```

- **Security Config**:

```
@Override protected void configure(HttpSecurity http) throws
Exception {   http
    .authorizeRequests()
      .antMatchers("/admin/**").hasRole("ADMIN")
      .antMatchers("/user/**").hasRole("USER")
      .anyRequest().authenticated()
    .and()
    .httpBasic();
} @Bean public UserDetailsService userDetailsService(UserRepository
repo) {   return username -> {
    User user = repo.findByUsername(username)
      .orElseThrow(() -> new UsernameNotFoundException("User not found"));     return new
org.springframework.security.core.userdetails.User(       user.getUsername(),
user.getPassword(),
user.getRoles().stream().map(SimpleGrantedAuthority::new).collect(Collectors.toList())
    );
  };
}
```

Microservices Architecture

Introduction

- **Monolithic vs Microservices**:
  - Monolithic: Single app, all components tightly coupled.
  - Microservices: Independent services, loosely coupled, communicating via APIs.
- **Benefits**:
  - Scalability: Scale individual services.
  - Flexibility: Use different tech stacks.
  - Resilience: Failure in one service doesn't crash others.

Setting Up Microservices

- **Eureka Server**:
  - Dependency:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>   <artifactId>spring-cloud-starter-
netflix-eureka-server</artifactId>
</dependency>
```

  - Application:

```
@SpringBootApplication @EnableEurekaServer
public class EurekaServerApplication {
public static void main(String[] args) {
    SpringApplication.run(EurekaServerApplication.class, args);
  }
}
```

- Config:

```
server.port=8761 eureka.client.register-with-
eureka=false eureka.client.fetch-registry=false
```

- **Eureka Client**:
  - Dependency:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>   <artifactId>spring-cloud-starter-
netflix-eureka-client</artifactId>
</dependency>
```

  - Config:

```
spring.application.name=user-service eureka.client.service-
url.defaultZone=http://localhost:8761/eureka
```

  - Application:

```
@SpringBootApplication
@EnableDiscoveryClient public class
UserServiceApplication {   public static
void main(String[] args) {
    SpringApplication.run(UserServiceApplication.class, args);
  }
}
```

Communication

- **REST**:

```
@RestController public class
UserController {
  @Autowired   private RestTemplate restTemplate;   @GetMapping("/call-product")   public
String callProductService() {     return restTemplate.getForObject("http://product-
service/api/products", String.class);
  }
}
```

- **Feign Client**:

```
@FeignClient(name = "product-service") public
interface ProductClient {
  @GetMapping("/api/products")
  String getProducts();
}
@RestController public class
UserController {
  @Autowired   private ProductClient
productClient;   @GetMapping("/call-
product")   public String callProduct()
{     return
productClient.getProducts();
  }
}
```

Resilience Patterns

- **Circuit Breaker with Resilience4j**:
  - Dependency:

```
<dependency>
```

```
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot2</artifactId>
</dependency>
```

- Config:

```
resilience4j.circuitbreaker.instances.productService.sliding-window-size=10
resilience4j.circuitbreaker.instances.productService.failure-rate-threshold=50
```

- Usage:

```
@RestController public class
UserController {
  @Autowired   private ProductClient
productClient;
  @GetMapping("/call-product")
  @CircuitBreaker(name = "productService", fallbackMethod = "fallback")
public String callProduct() {     return productClient.getProducts();
  }   public String fallback(Throwable t)
{     return "Product service
unavailable";
  }
}
```